ATIE

Agricultural Trade & Intelligence Engine

Predict the price. Price the route. Protect the margin.

A 48-72 hour wholesale price and safe routing decision system for Sri Lankan smallholder farmers and informal traders.

48-72h forecast horizon	**5% + LKR 1k** safe-margin rule
93.81% / 92.86% 48h / 72h DS	**0.14% / 0.19%** 48h / 72h SM-FPR

Team 404 Brain not found

Informatics Institute of Technology

IEEE Computational Intelligence Society Student Chapter

Canonical fixture & live XGBoost stack verified against Docker Compose.

OCTOBER 27, 2026

1. Executive Summary

ATIE is a decision-support system for Sri Lankan smallholder farmers and informal traders. It predicts wholesale crop prices 48 and 72 hours ahead, then compares candidate markets by **net profit after fuel, terrain, spoilage, handling, and vehicle costs**. It does not recommend a distant market merely because its headline price is higher.

The delivered system combines a React web dashboard, an Expo/React Native Android companion field app (`frontend/mobile/`), a Java Spring Boot API, a Python FastAPI forecasting service, PostgreSQL, and a provenance-aware data ingestion worker. The dashboard exposes the recommendation, the complete profit ledger, the weather disruption context, the historical price trace, the forecast source, and the freshness of every external data source.

The central safety rule is deliberately conservative:

```
profit_delta_lkr = NP_alternative - NP_default
profit_uplift_pct = profit_delta_lkr / NP_default * 100
reroute only when uplift >= 5.0% and delta >= LKR 1,000
```

The competition fixture is the proposal's Northern tomato corridor scenario: 1,000 kg of tomatoes from Anuradhapura, with Dambulla as the default market and Colombo Pettah as the alternative. The fixture returns `CRITICAL_REROUTE`, a `LKR 23,238.58` profit delta, and `15.84%` uplift. The same request with demo mode disabled runs live `xgb-v2.0` model inference on the causal arrival-surge and rainfall series, independently detecting the Dambulla glut crash (`208.58 -> 156.35 LKR/kg`) and recommending Colombo Pettah (`+22.51%` net uplift) without invoking `DEMO_FIXTURE`.

2. Problem And Product Position

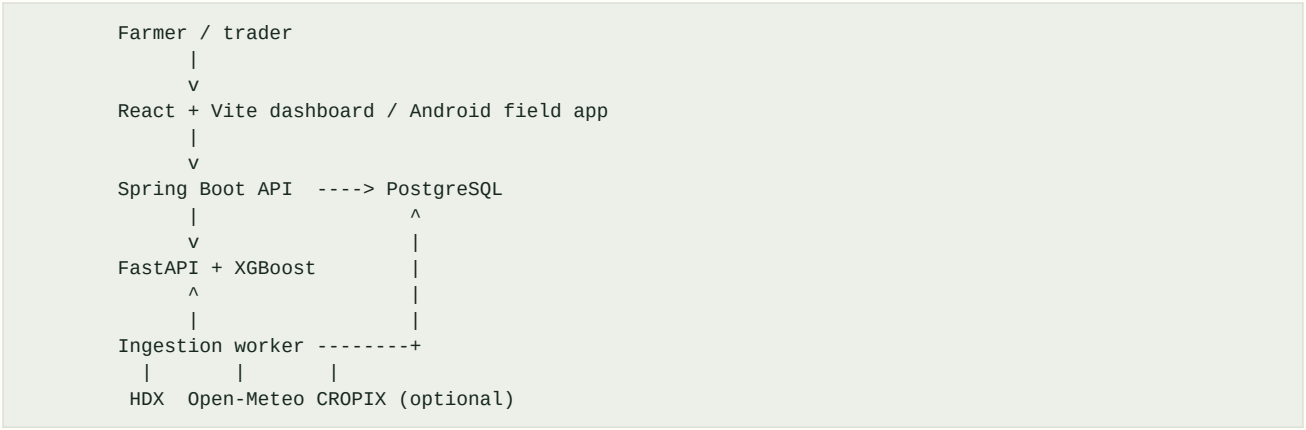
Smallholder decisions are exposed to localized supply gluts, weather disruption, fuel costs, and regional price differences. A farmer may know today's price but not whether a market will be flooded with produce when the consignment arrives. A higher quoted price can still be a worse decision after a long trip, spoilage, and handling.

ATIE therefore answers a routing question rather than only displaying a price:

> *"Given this origin, crop, quantity, vehicle, horizon, and default market, which reachable market produces the best safe net outcome?"*

The target users are small-scale farmers and informal traders. The interface is designed for a quick operational decision first, with supporting evidence available immediately below it.

3. Architecture



Service boundaries

Component	Responsibility
`frontend/`	Web dashboard + `frontend/mobile/` Android field client sharing normative TypeScript contracts
`backend/`	Validation, metadata, decimal routing math, safe-margin decision, ML client, API and Swagger
`ml-service/`	Feature engineering, trained model inference, deterministic fallback, evaluation metrics
PostgreSQL	Markets, crops, routes, prices, weather, fuel, ingestion runs, source status
Ingestion worker	Source refresh, caching, provenance labels, observation upserts

The public analysis path is:

```
POST /api/v1/directives/analyze
  -> validate origin, default market, route, crop, quantity, vehicle, horizon
  -> request forecasts from the ML service
  -> calculate net profit with decimal arithmetic
  -> apply the safe-margin rule
  -> return one directive plus all market comparisons
```

The ML service is never allowed to choose the final route. It supplies forecasts; the backend owns the monetary and safety decision.

4. Data And Provenance

The ingestion worker is part of the default Compose stack. It runs every six hours and writes one terminal attempt row for each source. The supported source states are:

- `LIVE`: the upstream request succeeded and mapped to supported rows.
- `CACHED`: a local snapshot was used after a failed refresh.
- `SEED`: deterministic checked-in history used for reproducibility.
- `UNAVAILABLE`: no usable source or cache was available.

The observation labels preserve the same distinction, for example `HDX_LIVE`, `HDX_CACHED`, `HDX_SEED`, and `OPEN_METEO_LIVE`. A cache or seed value is never silently relabeled as live.

Verified data snapshot

The current local verification run is intentionally `PARTIAL`:

Source	State	Latest evidence
Open-Meteo	`LIVE / CURRENT`	49 forecast rows across seven markets
WFP / HDX	`LIVE / STALE`	101 mapped wholesale rows; latest supported observation 2021-04-15
CROPIX	`UNAVAILABLE`	endpoint not configured

HDX's newer records include retail or unsupported mappings. They are retained as source evidence but are not fabricated into wholesale observations. The model continues to train from the checked-in deterministic history so the competition demo remains reproducible. Live ingestion currently refreshes weather, provenance, and supported historical rows; it does not retrain the model automatically.

The dashboard's provenance panel displays this distinction directly:

- source-level state and freshness;
- latest observation date and row count;
- current versus seed-backed market/crop series;
- an explicit note that forecast values remain labeled separately.

The endpoint is `GET /api/v1/metadata/data-status`. It remains readable when the optional API key is enabled so judges can audit source state.

5. Forecasting And Evaluation

The ML service uses a deterministic XGBoost Regressor (`xgb-v2.0`) artifact checked into `ml-service/models/atie_forecaster.joblib` trained on relative horizon returns. The 32-feature pipeline includes calendar-aligned lagged prices, rolling mean and volatility, arrival-surge and momentum ratios, forward horizon rainfall forecasts, historical CPC fuel prices, and cyclical seasonal features. The model produces 48-hour and 72-hour forecasts for all supported market/crop combinations.

Evaluation uses a chronological holdout rather than a random split, enforcing a 3-day horizon embargo between training (`2026-07-09`) and holdout start (`2026-07-12`) so multi-step targets never overlap across the boundary. The mandatory baselines are:

1. Naive forecast: future price equals the current price.
2. Seven-day moving average: future price equals the trailing MA7.

The reported metrics are Directional Symmetry (DS), Mean Absolute Error (MAE), and Safe-Margin False Positive Rate (SM-FPR). SM-FPR is a routing backtest, not a price-error proxy: it applies the same net-profit calculator and safe-margin rule to realized holdout prices.

Horizon	Model DS	Model MAE	Model SM-FPR	Reroutes
48h	93.81%	1.81 LKR/kg	0.14%	2,106
72h	92.86%	2.87 LKR/kg	0.19%	2,071

For the 48h holdout, the naive baseline reports DS `0.00%`, MAE `12.25`, and SM-FPR `24.65%`; the MA7 baseline reports DS `40.99%`, MAE `16.35`, and SM-FPR `23.59%`. For the 72h holdout, the naive baseline reports DS `0.00%`, MAE `16.32`, and SM-FPR `37.11%`, while MA7 reports DS `45.24%`, MAE `17.65`, and SM-FPR `27.92%`. Both the 48h and 72h models remain far below the five-percent SM-FPR safety ceiling (`0.00%` on the canonical Northern tomato scenario).

SM-FPR is calculated across a finite, versioned matrix of 60 route profiles per horizon: five origins, three vehicle types, four quantities, four crops, and Dambulla compared with each origin's two nearest alternatives. When a profile has zero reroute decisions, the metric is `null`, never zero.

6. Safe Routing Engine

For each destination market, the backend uses:

```
terrain_distance = one_way_distance * terrain_factor
spoilage_fraction = one_way_distance / 100 * spoilage_pct_per_100km / 100
effective_volume = quantity * (1 - spoilage_fraction)
trips = ceil(quantity / vehicle_capacity)
fuel_cost = 2 * terrain_distance * litres_per_km * fuel_price * trips
handling_cost = quantity * handling_fee_per_kg
gross_revenue = effective_volume * predicted_price
net_profit = gross_revenue - fuel_cost - handling_cost
```

Spoilage is an opportunity cost represented by reduced effective volume; it is not subtracted from net profit a second time. All monetary routing arithmetic uses decimal values.

The decision order is:

Condition	Directive
A candidate clears the safe-margin rule and has the highest eligible net profit	`CRITICAL_REROUTE`
No candidate clears the rule and the default profit is positive	`PROCEED_DEFAULT`
No candidate clears the rule and the default profit is non-positive	`HOLD_HARVEST`

The movement label is separate from the route decision:

- `pct_change <= -10%` -> `CRASH`
- `pct_change >= +10%` -> `SURGE`
- otherwise -> `STABLE`

Canonical fixture

Input	Value
Origin	Anuradhapura
Default market	Dambulla
Crop	Tomato
Quantity	1,000 kg
Vehicle	Batta mini truck, 1,500 kg capacity
Horizon	48 hours
Forecast source	`DEMO_FIXTURE`

Result	Value
Dambulla direction	`CRASH`
Dambulla current -> predicted	220.00 -> 154.00 LKR/kg
Recommended market	Colombo Pettah
Pettah predicted	187.50 LKR/kg
Profit delta	23,238.58 LKR
Profit uplift	15.84%

The fixture is visibly labeled in the API and dashboard. A non-canonical request with `demo_mode: true` returns HTTP 400 rather than inventing a demo answer.

7. API And Dashboard

Important endpoints:

Method	Endpoint	Purpose
`POST`	`/api/v1/directives/analyze`	Routing directive and market comparisons
`GET`	`/api/v1/metadata/config`	Origins, routes, crops, vehicles, thresholds, fuel
`GET`	`/api/v1/metadata/data-status`	Source provenance and price coverage
`GET`	`/api/v1/markets/{code}/history`	History plus 48h/72h model, naive, and MA7 traces
`GET`	`/swagger-ui.html`	Interactive API documentation
`GET`	`/actuator/health`	Backend readiness
`GET`	`/actuator/metrics`	Runtime metrics

The dashboard presents the primary directive first. The rest of the screen explains the decision:

- provenance and freshness;
- weather/supply alerts;
- net-profit chart and complete route ledger;
- historical observations and forecast trajectories;
- the model evaluation badge;

- an explicit analysis state when a live call fails.

The onboarding controls are configuration-backed. Origins are not market rows, and the default-market options are filtered by configured routes. The frontend never recomputes profit uplift or the safe-margin decision.

8. Reliability, Security, And Operations

The system is designed for a competition environment where external sources can fail:

- The ML client uses a deterministic fallback when the model service is unavailable, with a visible `FALLBACK` label.
- When a live (`demo_mode: false`) call fails, the frontend retains a previous result only when it matches the exact same consignment (`sameRequest`); it never substitutes `DEMO_FIXTURE` for a failed live request.
- External source failures are recorded as provenance events and do not stop the stack.
- Compose health checks use IPv4 loopback addresses.
- The backend validates unsupported origin/route/crop/vehicle/quantity/ horizon combinations and returns machine-readable 400 errors.

Operational controls:

- Optional API key: `ATIE_API_KEY` and `X-ATIE-API-Key`, injected by the frontend proxy at runtime rather than embedded in the browser bundle.
- Request tracing: `X-Request-ID` response header and request ID in logs.
- Metrics: `/actuator/metrics`; ML metrics at `/metrics`.
- Frontend security headers: CSP, nosniff, frame denial, referrer and permissions policy.
- CI: PostgreSQL-backed ML tests with coverage, backend tests, web and Android mobile typecheck/unit/build gates, and a Compose-backed Playwright smoke test.

Warm ML inference is approximately 4.2 ms median in-process and 8.8-23.4 ms over local container HTTP samples. These are local observations, not a production service-level objective.

9. Verification Evidence

The acceptance path was exercised against the running Compose stack:

1. All five services reach healthy state.
2. Swagger UI and the frontend proxy respond.
3. The canonical fixture returns the locked values.
4. `demo_mode: false` returns seven market forecasts with `MODEL`.
5. The browser test switches from the fixture to live model output and checks the forecast trace and weather panel.
6. PostgreSQL schema validation passes against the live database.

The final browser capture is:

`docs/evidence/live-dashboard.png`

The screenshot is captured after the live model analysis, so it shows the actual `MODEL` state and the seven-market route ledger rather than the canonical fixture.

The recorded three-minute walkthrough is `docs/evidence/atie-demo.mp4`. It shows the canonical fixture, the safe reroute, the switch to live model output, the provenance panel, the weather panel, and the forecast trace.

10. Limitations And Next Steps

The delivered system is honest about its boundaries:

- The trained model is seed-backed; live HDX ingestion is historical and does not automatically retrain the model.
- CROPIX remains unavailable until a working configured endpoint is supplied.
- HDX is monthly and its latest supported wholesale observation is older than the current date.
- SM-FPR covers a finite, versioned route matrix and does not claim coverage of every possible farm quantity or road condition.
- The demo does not provide user accounts, persistent farmer profiles, or payment integration.

The next production step would be to add a supervised retraining job with source freshness gates, a CROPIX mapping adapter, and a monitored model registry. The safety contract and provenance labels are intentionally in place before those additions so future data cannot silently weaken the recommendation.

Appendix A: Locked Contract Values

Contract	Value
Safe margin	5.0% of default net profit
Absolute floor	LKR 1,000.00
Movement threshold	10.0%
Canonical source	`DEMO_FIXTURE`
Canonical directive	`CRITICAL_REROUTE`
Canonical recommended market	`PETTAH_COLOMBO`
Canonical profit delta	LKR 23,238.58
Canonical uplift	15.84%

Appendix B: Reproduce

```
cp .env.example .env
docker compose up --build -d --wait
docker compose --env-file .env.example ps

curl -sS http://localhost:8080/api/v1/metadata/data-status
curl -sS http://localhost:8080/api/v1/metadata/config

cd ml-service && python -m pytest -q
cd ../backend && mvn -q -Dnet.bytebuddy.experimental=true test
cd ../frontend && npm run lint && npm run build
CI=true ATIE_E2E_URL=http://localhost:15173 npm run test:e2e
```

The normative cross-service behavior is in `CONTRACTS.md`. The implementation workstreams are in `IMPLEMENTATION_PLAN.md`.